

DEVALIVE

Backend Architecture Document (BAD)

Version: 1.0

Prepared By: Tanmay Bonde

Backend Stack:

- Node.js
- Express.js
- MongoDB Atlas
- Mongoose
- JWT
- Node Cron
- Nodemailer

Document Status: Approved

1. Introduction

Purpose

This document defines the complete backend architecture of DevAlive.

It serves as the blueprint for:

- Backend Development
- API Development
- Monitoring Engine Development
- Scheduler Development
- Notification Services
- Future Scalability

The architecture follows modern SaaS development standards with emphasis on:

- Scalability
- Maintainability
- Security
- Modularity
- Testability

2. Backend Architecture Overview

DevAlive backend consists of:

```
Client Requests
  ↓
Routes Layer
  ↓
Controllers Layer
  ↓
Services Layer
  ↓
Repositories Layer
  ↓
Database Layer
```

Supporting Systems:

```
Scheduler
Monitoring Engine
Notification Service
Authentication System
Logging System
Analytics Engine
```

3. Architectural Pattern

DevAlive follows:

Layered Architecture

```
Routes
  ↓
Controllers
  ↓
Services
  ↓
Repositories
```

↓
MongoDB

Benefits:

- Separation Of Concerns
- Reusable Logic
- Easier Testing
- Better Scalability
- Cleaner Codebase

4. Backend Folder Structure

```
backend/  
|  
├─ src/  
|  
├─ config/  
|   ├── db.js  
|   └── env.js  
|  
├─ routes/  
|   ├── auth.routes.js  
|   ├── project.routes.js  
|   ├── monitoring.routes.js  
|   ├── analytics.routes.js  
|   ├── notification.routes.js  
|   └── settings.routes.js  
|  
├─ controllers/  
|   ├── auth.controller.js  
|   ├── project.controller.js  
|   ├── monitoring.controller.js  
|   ├── analytics.controller.js  
|   ├── notification.controller.js  
|   └── settings.controller.js  
|  
├─ services/  
|   ├── auth.service.js  
|   ├── project.service.js  
|   ├── monitoring.service.js  
|   ├── analytics.service.js  
|   └── notification.service.js
```

```
|   |─ scheduler.service.js
|
|─ repositories/
|   |─ user.repository.js
|   |─ project.repository.js
|   |─ monitoring.repository.js
|   |─ notification.repository.js
|   |─ settings.repository.js
|
|─ models/
|   |─ User.js
|   |─ Project.js
|   |─ MonitoringLog.js
|   |─ Notification.js
|   |─ UserSettings.js
|
|─ middleware/
|   |─ auth.middleware.js
|   |─ validation.middleware.js
|   |─ error.middleware.js
|   |─ rateLimit.middleware.js
|
|─ validators/
|   |─ auth.validator.js
|   |─ project.validator.js
|
|─ jobs/
|   |─ monitoring.job.js
|
|─ utils/
|   |─ generateToken.js
|   |─ responseHandler.js
|   |─ logger.js
|
|─ constants/
|
|─ logs/
|
|─ app.js
|─ server.js
```

5. Layer Responsibilities

Routes Layer

Purpose:

Define API endpoints.

Responsibilities:

- URL mapping
- Middleware assignment
- Request routing

No business logic allowed.

Example:

```
POST /auth/register  
GET /projects  
POST /projects
```

Controllers Layer

Purpose:

Handle request and response flow.

Responsibilities:

- Receive request
- Validate inputs
- Call services
- Return responses

Controllers should remain thin.

No database logic allowed.

Services Layer

Purpose:

Contains business logic.

Responsibilities:

- Monitoring execution
- Analytics calculations
- Authentication logic
- Notification logic

This is the heart of the backend.

Repository Layer

Purpose:

Database abstraction layer.

Responsibilities:

- Database queries
- Data retrieval
- Data persistence

Benefits:

- Easier testing
 - Easier database migration
 - Cleaner services
-

Database Layer

Technology:

MongoDB Atlas

Responsibilities:

- Data storage
- Indexing
- Query execution

6. Authentication Architecture

Authentication Method

JWT Authentication

Password Encryption

bcrypt

Flow

```
Register
↓
Hash Password
↓
Store User

Login
↓
Verify Password
↓
Generate JWT
↓
Return Token
```

Protected Routes

Require JWT Middleware

7. Monitoring Engine Architecture

Purpose

Core business engine of DevAlive.

Responsible for:

- Health Checks
 - Status Validation
 - Response Tracking
 - Failure Detection
-

Monitoring Execution Flow

```
graph TD; A[Scheduler Trigger] --> B[Get Active Projects]; B --> C[Send HTTP Request]; C --> D[Receive Response]; D --> E[Analyze Result]; E --> F[Save Monitoring Log]; F --> G[Update Project Stats]; G --> H[Trigger Notification];
```

Monitoring States

```
SUCCESS

FAILURE

TIMEOUT

ERROR
```

8. Scheduler Architecture

Technology

Node Cron

Purpose

Execute monitoring cycles automatically.

Scheduler Flow

```
graph TD; A[Application Starts] --> B[Scheduler Starts]; B --> C[Cron Trigger]; C --> D[Monitoring Service]; D --> E[Execute Checks];
```

MVP Strategy

Node Cron

Future Strategy

```
graph TD; A[BullMQ] --> B[Redis Queue]; B --> C[Worker Nodes];
```

Architecture is designed for future migration.

9. Notification Architecture

Purpose

Alert users about project issues.

Technology

Nodemailer
SMTP

Notification Types

Downtime Alert
Recovery Alert
Monitoring Failure
System Notification

Notification Flow

Failure Detected
↓
Create Notification
↓
Send Email
↓
Store Notification Record

10. Analytics Engine Architecture

Purpose

Generate dashboard statistics.

Responsibilities

- Uptime Calculation
 - Failure Statistics
 - Response Time Analytics
 - Monitoring Summaries
-

Optimization Strategy

Analytics values stored in Project documents.

Examples:

```
uptimePercentage  
totalChecks  
successfulChecks  
failedChecks  
averageResponseTime
```

Reduces expensive database calculations.

11. Middleware Architecture

Authentication Middleware

Responsibilities

- Verify JWT
 - Extract User
 - Protect Routes
-

Validation Middleware

Responsibilities

- Validate Request Body
 - Validate Params
 - Validate Query
-

Rate Limit Middleware

Responsibilities

Protect:

Login
Register
Manual Health Check

Error Middleware

Responsibilities

- Catch Exceptions
- Format Errors
- Return Standard Response

12. Logging Architecture

Three Log Categories

Application Logs

Purpose:

General backend events.

Monitoring Logs

Purpose:

Health-check history.

Stored in database.

Error Logs

Purpose:

Debugging and investigation.

Future

Winston
Morgan
Cloud Logging

13. Error Handling Strategy

Centralized Error Handling

Pattern

```
Controller
↓
Service
↓
Throw Error
↓
Global Error Handler
```

Standard Error Response

```
{
  "success": false,
  "message": "Error Message"
}
```

14. Security Architecture

Authentication

JWT

Password Security

bcrypt

Request Validation

express-validator

Security Middleware

Helmet

Rate Limiting

express-rate-limit

Environment Security

dotenv

Never expose:

- JWT Secret
 - Database URI
 - SMTP Credentials
-

15. Performance Optimization Strategy

Pagination

Used For:

Projects

Logs

Notifications

Database Indexing

Indexed Fields

email

userId

projectId

status

checkedAt

Analytics Caching Strategy

Store calculated statistics.

Avoid repeated aggregation.

16. Scalability Strategy

Current MVP

Single Backend Server

Future

Load Balancer

↓

Multiple API Servers

↓

Redis Queue



Monitoring Workers

Future Enhancements

- Horizontal Scaling
 - Distributed Monitoring
 - Queue Processing
 - Multi Region Monitoring
-

17. Development Standards

Controllers

Maximum responsibility:

Request + Response

Services

Maximum responsibility:

Business Logic

Repositories

Maximum responsibility:

Database Operations

Rules

- No DB Queries In Controllers

- No Business Logic In Routes
 - No Response Logic In Services
-

18. Backend Module Map

Authentication Module

Auth Routes
Auth Controller
Auth Service
User Repository

Project Module

Project Routes
Project Controller
Project Service
Project Repository

Monitoring Module

Monitoring Routes
Monitoring Controller
Monitoring Service
Monitoring Job
Monitoring Repository

Analytics Module

Analytics Routes
Analytics Controller
Analytics Service
Monitoring Repository

Notification Module

```
Notification Routes
Notification Controller
Notification Service
Notification Repository
```

19. Backend Architecture Summary

The DevAlive backend is designed as a modular SaaS architecture built on:

- Node.js
- Express.js
- MongoDB Atlas
- JWT Authentication
- Node Cron Scheduling
- Nodemailer Notifications

The architecture emphasizes:

- Scalability
- Security
- Maintainability
- Testability
- Future Enterprise Expansion

while keeping the MVP implementation lightweight and developer-friendly.